SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

Somaiya TRUST

EdgeFall

On-Device Fall Detection from Phone Accelerometer and Gyroscope Data Using a Compressed 1D-CNN

Edge AI Lab CA Mini Project Report

Submitted by

Team 14

Pradyum Mistry
Rayan Kokate
Aryan Raut
Satvik Gupta

B.Tech Computer Engineering, Last Year
K. J. Somaiya School of Engineering, Mumbai

Edge AI (Lab CA Mini Project), Semester VII, 2026-2027
October 2026

Code: github.com/altf4-games/EdgeFall
Download (Releases): github.com/altf4-games/EdgeFall/releases

Abstract

EdgeFall detects falls using only the accelerometer and gyroscope already inside a phone. The motion data never leaves the device. A small 1D convolutional network with 3,393 parameters looks at the last two seconds of movement every 100 ms and decides whether it resembles a fall.

We trained the network on SisFall, a public dataset of falls and everyday movements recorded from 38 volunteers [1]. Each recording is converted to physical units, reduced to acceleration magnitude, rotation magnitude and jerk, downsampled from 200 Hz to 50 Hz and cut into 2-second windows. After post-training quantization the model is 11,664 bytes (11.4 KB). On five volunteers who were held out of training, the model in the app catches 90% of fall windows at 59% precision with a 0.9 threshold. The app itself uses a stricter 0.95 threshold and needs two positive windows in a row, which gives 81% recall and 62% precision per window.

For the compression part of the course we ran a separate, seeded training run and compared it with its own float32 baseline. Quantization cut the TFLite file from 18.5 KB to 11.4 KB with no change in precision or recall. Structured pruning, which removed half the filters in each convolution layer, cut the parameters by 68% and, after fine-tuning, scored about the same F1 as the unpruned model (0.87 against 0.86). The main limits are that SisFall's falls were performed onto a mattress by people wearing a belt sensor, so a bare phone dropped on a bed can trigger a false alarm, and that we measured latency on a server CPU and not on the phone.

Keywords: *edge AI, fall detection, 1D-CNN, SisFall, TensorFlow Lite, quantization, structured pruning, on-device inference.*

Table of contents

1. Introduction & problem statement....6
1.1 Background....6
1.2 Motivation....6
1.3 Objectives....6
2. Hardware & environment specifications....7
2.1 Edge device specifications....7
2.2 Software & framework stack....7
3. Methodology & model optimization pipeline....8
3.1 Baseline model selection....8
3.2 Model compression techniques....10
3.2.1 Structured pruning....10
3.2.2 Quantization technique....11
3.3 Compilation & execution engine....11
4. Performance & comparison analysis....13
4.1 Baseline AI vs. Edge AI comparative matrix....13
4.2 Compression & memory reduction metrics....14
4.3 Latency & accuracy trade-offs....14
5. Conclusion & future scope....17
5.1 Summary of results....17
5.2 Future improvements....17
5.3 GitHub link....17
5.4 Google Drive link with code....17
References....18

List of figures

Figure 1: Preprocessing pipeline and 1D-CNN architecture....8
Figure 2: Loss, precision and recall per epoch from the seeded re-run (best epoch 5 of 13)....10
Figure 3: Inference flow on the phone....12
Figure 4: Size, latency and F1 for each variant (re-run model)....14
Figure 5: Row-normalized confusion matrices of the shipped model....15
Figure 6: ROC and precision-recall curves of the shipped model, with the three thresholds marked....15
Figure 7: Distribution of the shipped model's output on the test set (log scale)....16

List of tables

Table 1: Edge device used for EdgeFall....7
Table 2: Software and framework stack....7
Table 3: SisFall as used in this project....8
Table 4: Subject-wise split of the 27,708 windows....9
Table 5: Training configuration....10
Table 6: Effect of pruning half of the filters in each convolution layer....11
Table 7: Baseline AI compared with Edge AI (re-run model, threshold 0.9)....13
Table 8: Model size after each compression step (re-run model)....14
Table 9: Accuracy and latency of each variant at threshold 0.9 (re-run model)....14
Table 10: Shipped model on the 3,611 test windows at three thresholds....15

1. Introduction & problem statement

1.1 Background

Falls are one of the most common ways older people get hurt, and how quickly someone finds out about a fall often matters as much as the fall itself. Commercial fall detectors exist, but most are pendants or wristbands that the person has to buy, charge and remember to wear. Almost everyone already carries a phone, and every modern phone has an accelerometer and a gyroscope, the same two sensors used in published fall-detection research.

Machine learning can turn those two signals into a fall detector, but where the model runs matters. Sending raw motion data to a server needs a network connection, adds delay and means a stream of personal data leaves the phone. Edge AI keeps the model on the device. The price is that the model has to be small and cheap enough to run all day on a phone's CPU without draining the battery.

1.2 Motivation

We wanted a fall detector that works with no extra hardware, no account and no internet connection. Three things pushed us towards running it on the phone. The first is privacy: motion data can show when someone is home, asleep or walking, so it is better if it never leaves the device. The second is reliability: a phone in a lift or a basement may have no signal, and the alert should still fire. The third is cost, since a model that fits in about 11 KB can run continuously in the background, which a cloud round trip every 100 ms could not.

The project also suits this course, because the model has to be shrunk and measured and not only trained. Size, speed and accuracy all matter at the same time.

1.3 Objectives

1. Detect falls using only the phone's built-in accelerometer and gyroscope, with no wearable and no network.
2. Train a compact 1D-CNN on the SisFall dataset and evaluate it on volunteers who are not in the training data.
3. Apply model compression, structured pruning and quantization, and measure what each one saves and what it costs in accuracy.
4. Deploy the model as TensorFlow Lite in an Android app that keeps monitoring in the background and alerts locally.
5. Report accuracy, size and latency honestly, including where the results may not carry over to real use.

2. Hardware & environment specifications

2.1 Edge device specifications

EdgeFall needs no hardware beyond a phone. There is no board, wearable or external sensor, so there is no hardware setup to photograph or geotag for this report. The app was built and tested on a Nothing Phone (3a). Table 1 lists the parts of it that matter for this project.

Table 1: Edge device used for EdgeFall

Item	Specification
Device	Nothing Phone (3a)
Processor	Qualcomm Snapdragon 7s Gen 3 (4 nm), octa-core: 1 x 2.5 GHz, 3 x 2.4 GHz, 4 x 1.8 GHz [9]
Memory	8 GB or 12 GB RAM, depending on the variant [9]
Operating system	Android 15 with Nothing OS [9]
Sensors used	Accelerometer and gyroscope
Other sensors	Compass, light and proximity [10]. No barometer, which the project does not need
Where the model runs	CPU, through the LiteRT (TensorFlow Lite) runtime. No GPU or NPU delegate is configured
Sensor rate	Requested at the game interval, then read by a timer at a fixed 50 Hz
Model file	11,664 bytes, bundled in the APK as an asset

Training and all the measurements in Section 4 were done on a cloud Linux container with four vCPUs (Intel Xeon at 2.80 GHz), 15 GB of RAM and no GPU. The model is small enough that a GPU is not needed. This matters for the latency numbers later, because they come from that server CPU and not from the phone.

2.2 Software & framework stack

Table 2 lists the software. The Python side does the data preparation, training, compression and evaluation. The Flutter side is only the way the model reaches a phone: it reads the sensors [7], runs the model and shows notifications, so we describe it briefly.

Table 2: Software and framework stack

Layer	Tool	Version or note
Dataset	SisFall, community mirror [11]	4,505 recordings used
Language	Python	3.13
Numerics, metrics	NumPy, scikit-learn	NumPy 2.5.3, scikit-learn 1.9.1
Training	TensorFlow with Keras	TensorFlow 2.21.0, Keras 3.15.1
Plots	Matplotlib	3.11.2
Export	TensorFlow Lite converter	Part of TensorFlow 2.21.0
App framework	Flutter (Dart)	Dart SDK ^3.13.1
Sensor access	sensors_plus	^6.1.1
Background monitoring	flutter_background_service	^5.1.0
Alerts	flutter_local_notifications	^18.0.1
Inference	flutter_litert (LiteRT bindings) [6]	^3.8.0

3. Methodology & model optimization pipeline

The pipeline has two halves, shown in Figure 1. Data preparation turns raw SisFall files into normalized windows, and a small 1D-CNN turns each window into a fall probability. Everything after that (pruning, quantization and conversion) is about making the trained network cheaper to run.

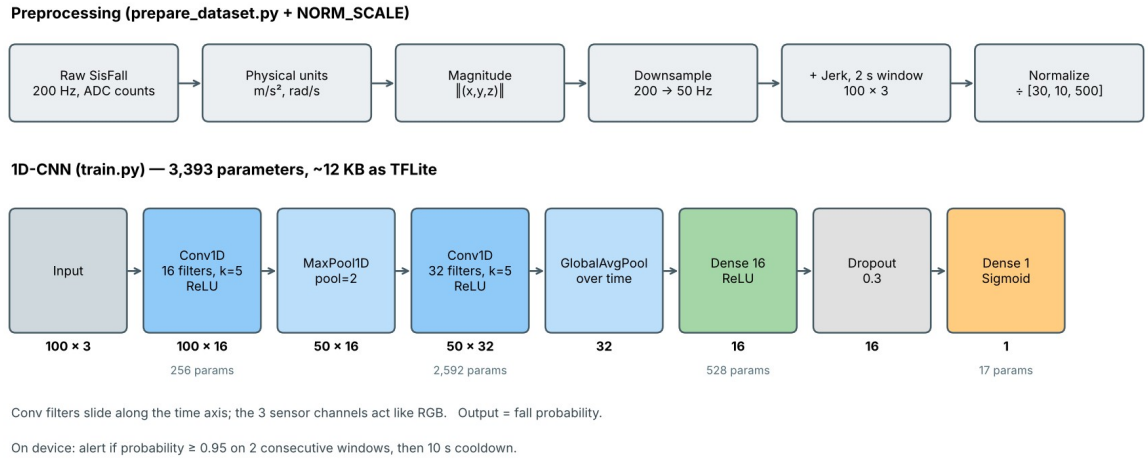


Figure 1: Preprocessing pipeline and 1D-CNN architecture

3.1 Baseline model selection

Dataset

We used SisFall [1]. It has recordings from 38 volunteers: 23 young adults (IDs starting with SA, aged 19 to 30) and 15 elderly people (SE, aged 60 to 75). Each volunteer performed 19 kinds of everyday movement, called ADLs (activities of daily living), such as walking, jogging, climbing stairs, sitting down and getting into a car. The young adults also performed 15 kinds of fall, among them slips, trips and fainting while walking, and falls while sitting down or getting up. The only elderly volunteer who simulated falls was SE06. The sensors were a waist-mounted ADXL345 accelerometer and an ITG3200 gyroscope sampled at 200 Hz. The dataset also has a second accelerometer (MMA8451Q) that we ignored, because its +-8 g range clips during hard impacts. The original host is offline, so we used a community mirror [11], which has 4,505 recording files.

Table 3: SisFall as used in this project

Item	Value
Volunteers	38 (23 young adults, 15 elderly)
Recording files	4,505 (2,707 ADL, 1,798 fall)
Activity types	19 ADL (D01 to D19) and 15 fall (F01 to F15)
Raw format	Nine columns of integer ADC counts at 200 Hz, no header row
Columns used	ADXL345 accelerometer (X, Y, Z) and ITG3200 gyroscope (X, Y, Z)
Windows produced	27,708 (1,798 fall and 25,910 ADL)

Preprocessing

Each recording goes through the same steps, in this order (prepare_dataset.py):

1. Convert raw counts to physical units: the accelerometer to m/s2 and the gyroscope to rad/s, using each sensor's range and resolution.

2. Take the magnitude of each three-axis signal. A phone in a pocket can be in any orientation, and magnitude does not depend on orientation, whereas separate X, Y and Z values would.
3. Downsample from 200 Hz to 50 Hz by keeping every fourth sample. This matches the rate at which the app reads the phone's sensors.
4. Compute jerk, the rate of change of acceleration magnitude, as a third channel. A fall has a sharp jump in acceleration at impact, and jerk makes that jump easy to see.
5. Cut 2-second windows of 100 samples. Each fall recording gives one window centred on its peak acceleration, which is the impact. Each ADL recording is cut into back-to-back windows with no overlap.
6. Divide each channel by a fixed constant ([30, 10, 500] for acceleration, rotation and jerk) so that all three sit on a similar scale.

We used fixed constants and not the dataset's mean and standard deviation, because SisFall's belt-mounted sensor and a phone in a pocket see different signals. The constants are rough upper limits that we chose by hand: about 3 g of acceleration, about 570 degrees per second of rotation, and roughly the 99th percentile of jerk in the data. We did not tune them. The same numbers are in the Dart code, because the model only works if the phone scales its input exactly as training did.

Splitting the data

We split by volunteer and not by window. If windows from the same person end up in both the training and test sets, the model can learn that person's movement style and the test score looks better than it should. With a subject-wise split the test set only holds people the model has never seen. A seeded shuffle (seed 42) gave the split in Table 4.

Table 4: Subject-wise split of the 27,708 windows

Set	Volunteers	Windows	Fall windows
Train	28	20,817	1,423
Validation	5	3,280	150
Test	5 (SA05, SA06, SA23, SE05, SE07)	3,611	225

SE05 and SE07 are elderly volunteers who only performed ADLs, so all 225 test falls come from three young adults (SA05, SA06 and SA23: 15 fall types x 5 trials each).

Why a 1D-CNN

A fall is a short pattern in a time series: acceleration drops towards zero while falling, spikes on impact, and then the body is still. A 1D convolution slides small filters along the time axis and learns to spot patterns like that wherever they occur in the window. The three channels play the role that red, green and blue play in an image. We picked a CNN over a recurrent network because it is smaller, processes all time steps at once and quantizes well. We did not train a recurrent model, so we cannot say how it would compare. We picked a learned model over a hand-written threshold rule (the first version of the app, still in the repository as fall_detector.dart) because a learned model can use the whole window of data and not only a few trigger conditions.

The network has two convolution layers with 16 and 32 filters of width 5, so each filter looks at 0.1 s of signal. A max-pooling layer between them halves the time axis. Global average pooling then collapses the time axis into 32 numbers, one per learned pattern, and a small dense layer with dropout turns those into one probability between 0 and 1. In total there are 3,393 parameters.

Training

We trained with the Adam optimizer [4] and the settings in Table 5. Falls are only 6.5% of the windows, so a model that always answered "no fall" would be right 93.5% of the time. The class weight makes a

missed fall cost about 14 times more than a false alarm during training. We stopped on validation recall because a missed fall is the worse mistake.

Table 5: Training configuration

Setting	Value
Optimizer	Adam, learning rate 0.001
Loss	Binary cross-entropy
Class weights	ADL 1.0, fall 13.6 (negatives divided by positives in the training set)
Batch size	64
Maximum epochs	50
Early stopping	Monitor validation recall, patience 8, restore best weights
Dropout	0.3
Random seed	42
Training hardware	CPU only

We trained the network twice with the same code. The first run produced the model that ships in the app, and its training history was not saved. For this report we ran training again with a fixed random seed and saved the history. We use that second run for the compression experiments and for the curves in Figure 2. Early stopping ended the second run after 13 epochs and restored the weights from epoch 5, which had the best validation recall. The validation loss is lower than the training loss because the training loss includes the class weights and the validation loss does not.

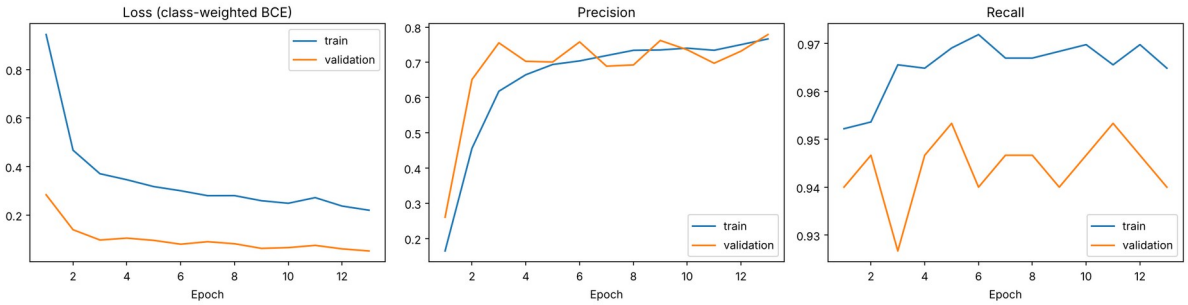


Figure 2: Loss, precision and recall per epoch from the seeded re-run (best epoch 5 of 13)

3.2 Model compression techniques

The baseline is already small, at 3,393 parameters. Compression here is less about making a huge model fit and more about seeing how much further a small one can shrink and what each step costs. We tried two techniques, structured pruning and post-training quantization. All numbers in this section and in Section 4 come from compression_experiments.py, which starts from the seeded re-run's float32 model and is evaluated on the same 3,611 test windows.

3.2.1 Structured pruning

Pruning removes the parts of a network that contribute little. Unstructured pruning zeroes individual weights, which leaves every layer the same shape and gives no speed-up on ordinary hardware unless special sparse kernels exist. Structured pruning removes whole filters, so the layers really get smaller. We used the L1-norm method of Li et al. [2]: rank each filter by the sum of the absolute values of its weights, and drop the filters with the smallest sums.

We kept half of the filters in each convolution layer: 8 of 16 in the first and 16 of 32 in the second. Removing a filter from the first layer also removes the matching input channel of the second layer, and removing one from the second layer removes the matching input row of the dense layer, so we sliced those weights as well. Then we fine-tuned the smaller network with the same recipe as before (class weights and early stopping on validation recall).

Table 6: Effect of pruning half of the filters in each convolution layer

Layer	Original (parameters)	Pruned (parameters)
Conv1D, first	16 filters (256)	8 filters (128)
Conv1D, second	32 filters (2,592)	16 filters (656)
Dense 16	32 inputs (528)	16 inputs (272)
Output	16 inputs (17)	16 inputs (17)
Total	3,393	1,073 (-68.4%)

Right after pruning, before any fine-tuning, the model still ranked windows reasonably well (ROC AUC 0.936 against 0.971), but its scores shifted: at the 0.9 threshold precision fell to 0.39 while recall stayed at 0.93. Fine-tuning fixed that. The fine-tuned pruned model scored precision 0.87, recall 0.87 and F1 0.869 at the same threshold, against 0.88, 0.84 and 0.858 for the unpruned model (Table 9). We would not read the small gain as pruning improving the model. The test set has only 225 falls and this is a single run, so differences of a point or two are within noise. What we can say is that half the filters were not needed for this task. We did not put the pruned model in the app, which still uses the unpruned quantized model.

3.2.2 Quantization technique

Quantization stores numbers with fewer bits. We used post-training dynamic-range quantization [3][5]. After training, the converter stores the weights as 8-bit integers and converts them back to float while computing, and the activations stay in floating point. It needs no calibration data and no retraining, and it is one line in the converter:

```
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
tflite_model = converter.convert()
```

The file shrank from 18.5 KB to 11.4 KB, a 38.3% reduction, and the outputs barely moved. At the 0.9 threshold precision, recall and F1 were identical to the float32 TFLite model, and ROC AUC differed only in the fifth decimal place (0.97091 against 0.97092). The saving is smaller than the 4x that 8-bit weights suggest. As far as we can tell, the converter leaves weight tensors with fewer than 1,024 elements in float, because the saving is too small to justify the conversion work. In this network only the second convolution layer (5 x 16 x 32 = 2,560 weights) is large enough. The pruned model supports this reading: none of its tensors reaches 1,024 elements, and its quantized and float files are exactly the same size (9.4 KB).

3.3 Compilation & execution engine

There is no compilation step that produces machine code. The converter turns the Keras graph into a TensorFlow Lite flatbuffer (fall_model.tflite) and the app runs it with an interpreter. The file is declared as a Flutter asset and goes into the APK. At start-up the app loads it with `Interpreter.fromAsset` from flutter_litert, a binding to the LiteRT (TensorFlow Lite) runtime [6]. The input tensor is 1 x 100 x 3

floats and the output is one float. The app does not configure a GPU or NNAPI delegate, so inference runs on the CPU with the library's defaults. For a model this small that is enough.

Figure 3 shows what happens on the phone. A foreground service keeps monitoring running when the app is closed. Every 20 ms a timer reads the latest accelerometer and gyroscope values, computes magnitudes and jerk, scales them with the same constants as training and pushes the sample into a buffer of the last 100. Once the buffer is full, the app runs the model on every fifth new sample, which is about every 100 ms. An alert needs the probability to be above 0.95 on two consecutive windows. After an alert the detector stays silent for 10 seconds, so a single fall, which spans many overlapping windows, does not raise a string of notifications. The alert is a local notification and nothing is sent anywhere.

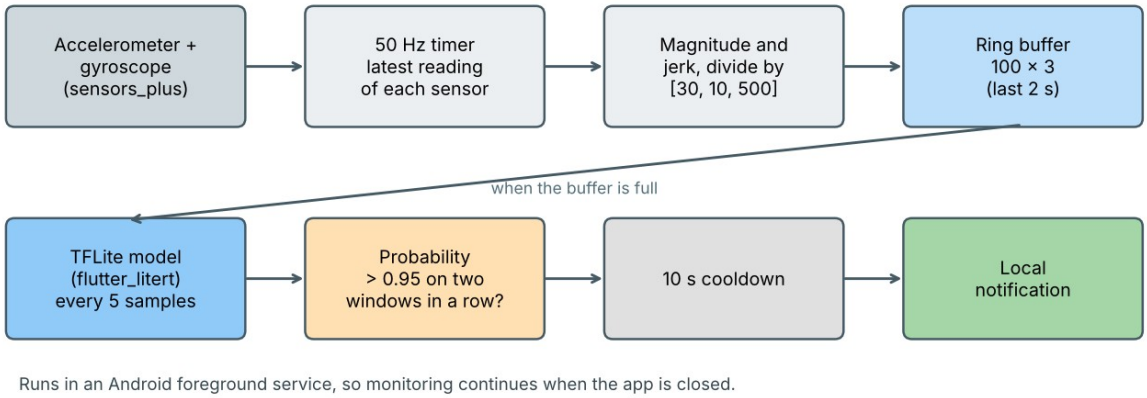


Figure 3: Inference flow on the phone

4. Performance & comparison analysis

All results use the same test set: 3,611 windows from five volunteers who were never used for training or for deciding when to stop (225 fall windows and 3,386 ADL windows). Two trained models appear in this section and we keep them apart on purpose. The *shipped* model is the file in the app (11,664 bytes). The *re-run* model is the seeded retraining that we compressed and timed. With the same code and the same split, the re-run scores higher than the shipped model (F1 0.86 against 0.72 at the 0.9 threshold). We take that as a sign that results vary from one training run to the next. We did not measure that spread over several seeds, and the shipped model's numbers should not be read next to the re-run's as if they came from one model.

4.1 Baseline AI vs. Edge AI comparative matrix

The baseline is the float32 Keras model running in Python on a server CPU, which is how the model would be used with no edge optimization. The edge version is the quantized TFLite model that runs inside the app. Both come from the re-run, so the differences in Table 7 come from the file format and the runtime and not from training.

Table 7: Baseline AI compared with Edge AI (re-run model, threshold 0.9)

Metric	Baseline AI: Keras float32	Edge AI: TFLite, dynamic-range quantized
Runtime	Python with TensorFlow	LiteRT interpreter inside the Android app
Model file	79.9 KB (.keras)	11.4 KB (.tflite)
Parameters	3,393	3,393
Weight storage	32-bit float	8-bit integer in the largest layer, float elsewhere
Precision	0.883	0.883
Recall	0.836	0.836
F1	0.858	0.858
ROC AUC	0.9709	0.9709
Median latency per window*	0.812 ms	0.0065 ms
Can run inside a phone app	No, needs Python and TensorFlow	Yes

*Median of 2,000 single-window runs on one CPU thread of the server described in Section 2.1. The Keras figure includes Python and TensorFlow call overhead, so the gap overstates how much faster the TFLite arithmetic is. The .keras file also stores the optimizer state and the model configuration, so its size is not a pure weight count. These are not measurements from the phone.

At the threshold we use, accuracy is identical across the two formats, so quantization has no visible accuracy cost on this task. The practical gain is that the edge model is a single 11.4 KB file that a phone can load with no Python runtime.

4.2 Compression & memory reduction metrics

Table 8: Model size after each compression step (re-run model)

Variant	Parameters	File size (KB)	vs. Keras file	vs. TFLite float32
Keras float32	3,393	79.9	baseline	n/a
TFLite float32	3,393	18.5	-76.9%	baseline
TFLite dynamic-range int8	3,393	11.4	-85.7%	-38.3%
Pruned, float32	1,073	9.4	-88.2%	-49.0%
Pruned, dynamic-range int8	1,073	9.4	-88.2%	-49.0%

Converting from the Keras file to a TFLite float32 file is the biggest single drop (76.9%), mostly because the .keras file carries training state. After that, quantization takes another 38.3% off and pruning takes 49.0% off. Pruning and quantization do not stack here, because of the 1,024-element rule from Section 3.2.2. The shipped model (11,664 bytes) is the quantized, unpruned variant. Runtime memory is mostly the 100 x 3 input buffer and the interpreter's own allocations, which we did not measure.

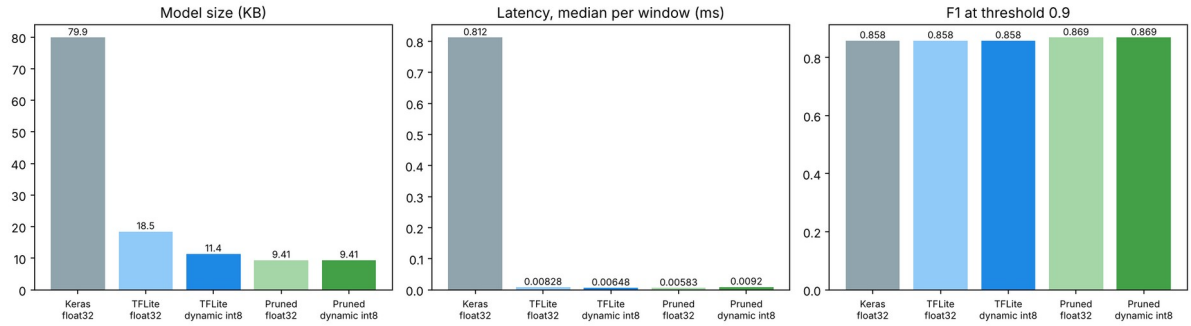


Figure 4: Size, latency and F1 for each variant (re-run model)

4.3 Latency & accuracy trade-offs

Table 9: Accuracy and latency of each variant at threshold 0.9 (re-run model)

Variant	Precision	Recall	F1	ROC AUC	Latency (ms)
Keras float32	0.883	0.836	0.858	0.9709	0.812
TFLite float32	0.883	0.836	0.858	0.9709	0.0083
TFLite dynamic int8	0.883	0.836	0.858	0.9709	0.0065
Pruned, before fine-tuning	0.388	0.929	0.547	0.9362	n/a
Pruned, float32	0.867	0.871	0.869	0.9705	0.0058
Pruned, dynamic int8	0.867	0.871	0.869	0.9705	0.0092

Every TFLite variant ran one window in under 0.01 ms on the server CPU, which is 6 to 9 microseconds. The differences among them are inside measurement noise: the pruned quantized model came out slower than the pruned float one even though the two are the same file size and structure. The app asks for an inference every 100 ms, so even a phone a hundred times slower than this server would spend a small fraction of its time on the model. We have not measured on the phone, and we expect the sensor timer, notifications and the foreground service to cost more battery than the model does, but we have not measured that either. A proper measurement on the Nothing Phone (3a) is the first item in Section 5.2.

Accuracy depends much more on the decision threshold than on compression. Table 10 shows the shipped model at three thresholds, and Figure 5 shows the same results as percentages of each true class.

Table 10: Shipped model on the 3,611 test windows at three thresholds

Threshold	Precision	Recall	Specificity	F1	TP / FP / FN / TN
0.5	0.448	0.956	0.922	0.610	215 / 265 / 10 / 3121
0.9	0.594	0.902	0.959	0.716	203 / 139 / 22 / 3247
0.95 (app)	0.617	0.809	0.967	0.700	182 / 113 / 43 / 3273

ROC AUC 0.968, average precision 0.504. Falls are 6.2% of the test windows (225 of 3,611).

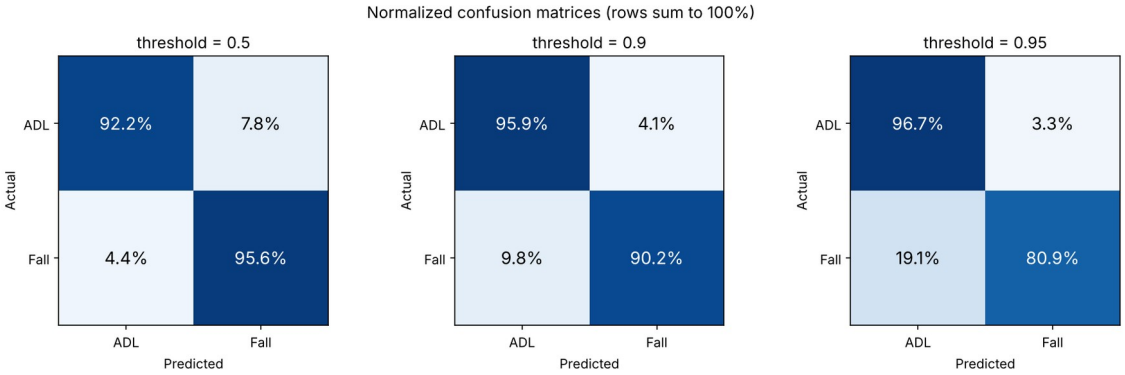


Figure 5: Row-normalized confusion matrices of the shipped model

The threshold moves the model along one curve. At 0.5 it finds 95.6% of falls but flags 7.8% of ordinary windows. At 0.95, which the app uses, it flags only 3.3% of ordinary windows but misses 19.1% of falls. We raised the app's threshold from 0.9 to 0.95 after trying it on the phone, because dropping the phone onto a soft surface set off alerts. The ROC curve in Figure 6 shows that the model separates the two classes well (AUC 0.968). The precision-recall curve explains why precision stays modest: only about 6% of test windows are falls, so even a 3.3% false-positive rate gives about two false alarms for every three real detections (113 against 182).

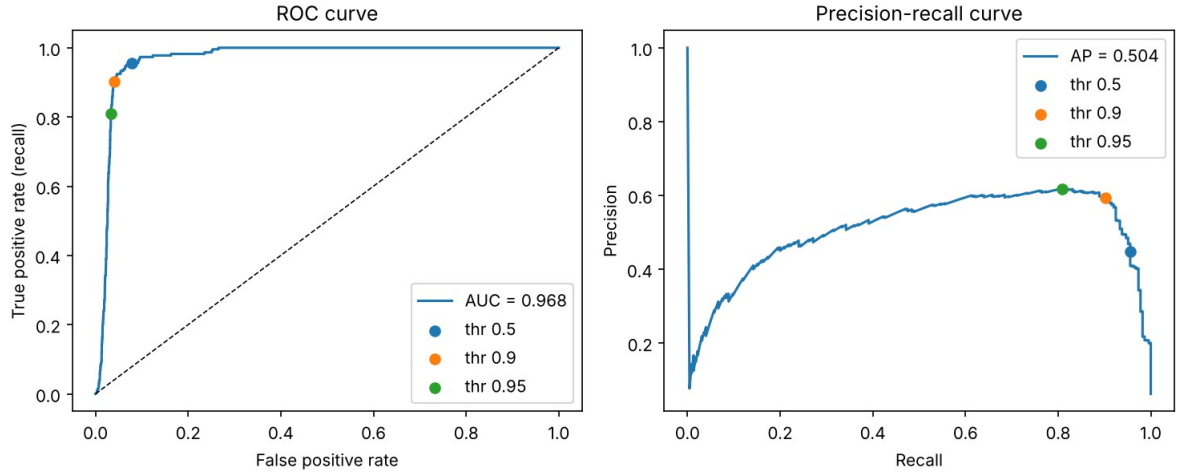


Figure 6: ROC and precision-recall curves of the shipped model, with the three thresholds marked

Figure 7 shows where the errors sit. Most ADL windows get a probability close to zero and most fall windows get one close to one, but the two overlap above 0.9, which is why the last few points of threshold trade so much recall for so little precision.

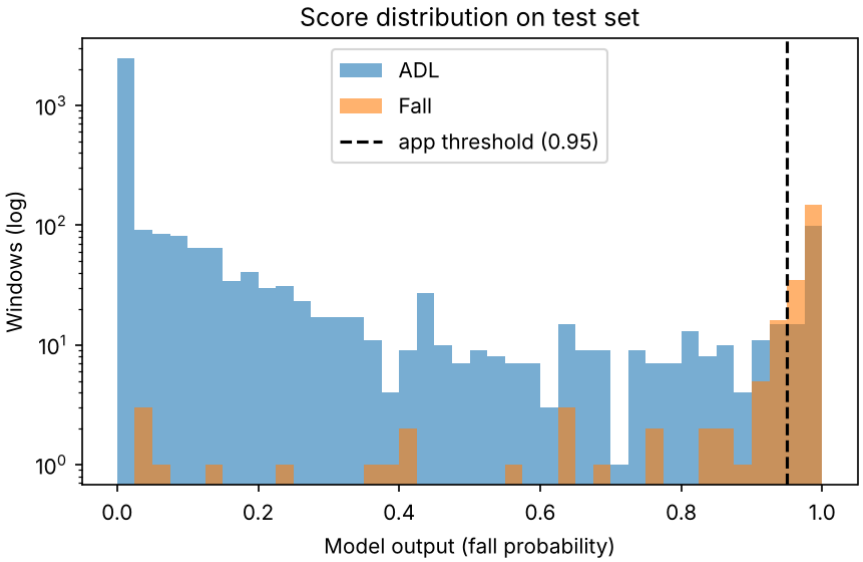


Figure 7: Distribution of the shipped model's output on the test set (log scale)

The two-consecutive-windows rule should lower false alarms further, because windows overlap heavily and a short spike is unlikely to repeat. We have not measured it. All the numbers above are per window on SisFall's trimmed clips, and an hour of a phone sitting in a pocket may look different. That is the biggest gap between these results and real use.

Four more things limit how far these results can be trusted. SisFall's falls were performed onto a mattress by volunteers wearing a belt sensor, whereas the app runs on a phone in a pocket or hand, so the signal differs, and a bare phone dropped on a bed or sofa can look like a cushioned fall. Elderly volunteers contributed no falls except SE06, who is not in the test set, so there is no result for real elderly falls. Each fall window is centred on the impact while ADL windows are cut without regard to their content, which makes the test easier than continuous monitoring. Finally, the compressed models were each trained once, and timed on a server CPU.

5. Conclusion & future scope

5.1 Summary of results

EdgeFall runs a 3,393-parameter 1D-CNN on a phone's accelerometer and gyroscope and raises a local notification when it sees a fall. The model in the app is 11,664 bytes. On five volunteers it never saw in training, it detects 90.2% of fall windows at 59.4% precision with a 0.9 threshold, and 80.9% at 61.7% with the 0.95 threshold the app uses. ROC AUC is 0.968.

In the compression experiments, quantization cut the TFLite file by 38.3% with no change in accuracy. Structured pruning removed 68.4% of the parameters and 49.0% of the file size and, after fine-tuning, kept the F1 score (0.869 against 0.858). The two did not add up, because the pruned layers are too small for the converter to quantize. The main weakness is false alarms from soft-surface drops, which follows from SisFall's padded falls and belt-worn sensor.

5.2 Future improvements

1. Measure latency, memory and battery use on the Nothing Phone (3a) and not on a server CPU.
2. Record a few hundred windows of a phone dropped on beds, sofas, tables and floors, and fine-tune on them as hard negatives. This goes straight at the main false-alarm source.
3. Evaluate on long continuous recordings, with falls hidden inside hours of ordinary movement, and report false alarms per hour for the full decision rule.
4. Train with several random seeds and report the spread before deciding which model to ship, since the re-run in this report scored higher than the shipped one.
5. Ship the pruned model if it holds up across seeds, and try quantization-aware training or full integer quantization if we move to hardware that needs it.

5.3 GitHub link

Source code, training scripts, results and this report's experiment scripts: https://github.com/altf4-games/EdgeFall

Download the Android app (APK) from the Releases section:
https://github.com/altf4-games/EdgeFall/releases (release v0.1.0:
https://github.com/altf4-games/EdgeFall/releases/tag/v0.1.0)

5.4 Google Drive link with code

Google Drive link: [paste the share link here]

References

[1] A. Sucerquia, J. D. López and J. F. Vargas-Bonilla, "SisFall: A Fall and Movement Dataset," Sensors, vol. 17, no. 1, article 198, 2017.

[2] H. Li, A. Kadav, I. Durdanovic, H. Samet and H. P. Graf, "Pruning Filters for Efficient ConvNets," in Proc. International Conference on Learning Representations (ICLR), 2017.

[3] B. Jacob et al., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," in Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2018.

[4] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in Proc. International Conference on Learning Representations (ICLR), 2015.

[5] Google AI Edge, "Post-training quantization," LiteRT documentation, https://ai.google.dev/edge/litert/models/post_training_quantization.

[6] flutter_litert package, https://pub.dev/packages/flutter_litert.

[7] sensors_plus package, https://pub.dev/packages/sensors_plus.

[8] P. Mistry, R. Kokate, A. Raut and S. Gupta, "EdgeFall," https://github.com/altf4-games/EdgeFall.

[9] GSMArena, "Nothing Phone (3a)," https://m.gsmarena.com/nothing_phone_(3a)-ampp-13672.php.

[10] 91mobiles, "Nothing Phone 3a specifications," https://www.91mobiles.com/nothing-phone-3a-price-in-india?ty=specs.

[11] BIng2325, "SisFall," community mirror of the dataset (GitHub repository), https://github.com/BIng2325/SisFall.